Proceed karte hain! Module 7 ke baad aapka retrieval system accurate toh ho gaya hai, lekin enterprise systems mein hamesha ek cheez aati hai—**Strict Business Rules**.
Aapka model chahe jitna bhi bada semantic genius ho, agar user ne bola ki *"Mujhe sirf 2025 ki reports dikhao"*, toh vector similarity se kaam nahi chalega. Humme metadata filtering ki zaroorat padegi.
Chaliye shuru karte hain **Module 8: Metadata Filtering (Structured + Unstructured Search)**.

Module 8: Metadata Filtering

1. Why this concept exists

Real-world enterprise data hamesha pure text nahi hota. Har text/document ke saath kuch attributes jude hote hain—jaise **Date Created, Author, Department, File Type, ya Region**.
Metadata filtering isiliye exist karta hai taaki hum semantic search ko in structured attributes ke saath lock kar sakein, jisse non-relevant documents search engine se pehle hi filter-out ho jayein.

2. Problem with traditional RAG (Semantic Overlap)

Traditional Vector Search pure semantic meaning par chalta hai.
Problem: Maan lo aapke paas teen documents hain:

1. HR Policy for **US Region** (2023) - *Text: "Maternity leave is 12 weeks."*
2. HR Policy for **India Region** (2025) - *Text: "Maternity leave is 26 weeks."*
3. HR Policy for **UK Region** (2026) - *Text: "Maternity leave is 52 weeks."*

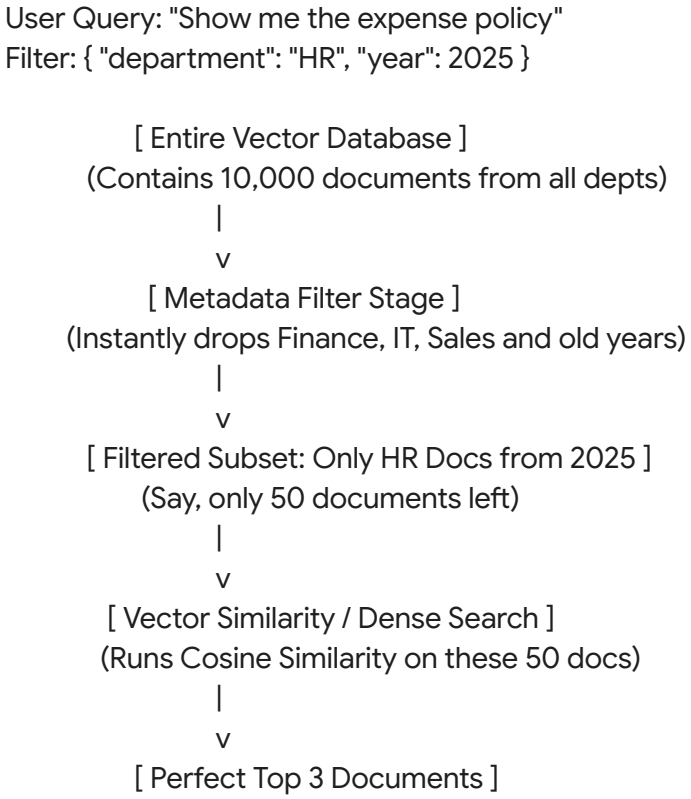
Agar ek Indian employee search kare: *"How many weeks of maternity leave do I get?"*, toh vector embedding teeno text ko semantically highly similar dikhayegi. Vector DB bina region check kiye top result mein UK ki policy return kar sakta hai, jo ki us employee ke liye 100% galat information hogi.

3. Real-world example

Aap ek Legal Document AI tool bana rahe hain kisi law firm ke liye.
Lawyer query karta hai: *"Find cases related to trademark infringement where the verdict was passed after 2024 and the department is Corporate Law."*
Yahan teen criteria hain:

1. Semantic concept: "Trademark infringement" -> (Vector/Dense Search)
2. Date rule: Year > 2024 -> (Metadata Filter)
3. Department rule: Department == Corporate Law -> (Metadata Filter)

4. Visual explanation using diagrams made with text



5. Architecture flow

Metadata filtering do tarike se ki ja sakti hai:

1. **Pre-filtering (Industry Standard):** Sabse pehle Vector DB relational database ki tarah metadata filtering query chalata hai (WHERE department = 'HR'). Isse search space drastically chota ho jata hai. Phir bache hue documents par vector similarity calculate hoti hai.
2. **Post-filtering:** Pehle top 100 vector matches nikale jaate hain, phir loop chalakar unme se check kiya jata hai ki kiska metadata rule match ho raha hai. (Yeh approach bad hai kyunki agar top 100 mein HR ka ek bhi doc nahi aaya, toh post-filtering empty result degi).

Modern DBs (FAISS, Chroma, Pinecone) natively Pre-filtering handle karte hain.

6. Step-by-step implementation

1. Documents load karte waqt unke metadata dictionary mein values populate karein (e.g., doc.metadata = {"source": "pdf", "year": 2025, "dept": "HR"}).
2. Vector store create karein.
3. Search run karte waqt LangChain ke search_kwargs mein ek special dictionary query pass karein jise filter kehte hain.
4. Filter syntax har Vector DB ke liye slightly alag ho sakta hai (Chroma and FAISS use standard dictionary mappings).

7. Complete Python code & LangChain implementation

Chaliye standard Chroma DB ke saath multi-level metadata filtering ka code dekhte hain.

```
import os
from langchain_core.documents import Document
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma

# Setup API Key
os.environ["OPENAI_API_KEY"] = "sk-your-openai-api-key"

# 1. Create Documents with rich Metadata
docs = [
    Document(
        page_content="Employees get 20 days of paid annual leaves.",
        metadata={"dept": "HR", "year": 2025, "file_type": "pdf"}
    ),
    Document(
        page_content="Maternity leave policy provides 26 weeks of paid time off.",
        metadata={"dept": "HR", "year": 2023, "file_type": "pdf"}
    ),
    Document(
        page_content="Server maintenance costs must be pre-approved by the CTO.",
        metadata={"dept": "IT", "year": 2025, "file_type": "docx"}
    ),
    Document(
        page_content="Travel reimbursement requires original food receipts and bills.",
        metadata={"dept": "Finance", "year": 2025, "file_type": "pdf"}
    )
]

# 2. Index into Vector Store
embeddings = OpenAIEmbeddings()
vectorstore = Chroma.from_documents(docs, embeddings)

# 3. Define the Search Query
```

```
query = "Tell me about time off and allowance policies."

# --- CASE 1: Filter Only HR Documents ---
print("--- CASE 1: Filtering strictly by dept='HR' ---")
hr_results = vectorstore.similarity_search(
    query,
    k=2,
    filter={"dept": "HR"}
)
for doc in hr_results:
    print(f"Content: {doc.page_content} | Metadata: {doc.metadata}")

# --- CASE 2: Advanced Logical Filtering ($and conditions) ---
# Chroma syntax uses MongoDB-style operators like $and, $eq, $gt
print("\n--- CASE 2: Filtering by dept='HR' AND year=2025 ---")
advanced_results = vectorstore.similarity_search(
    query,
    k=2,
    filter={
        "$and": [
            {"dept": {"$eq": "HR"}},
            {"year": {"$eq": 2025}}
        ]
    }
)
for doc in advanced_results:
    print(f"Content: {doc.page_content} | Metadata: {doc.metadata}")
```

8. Production use case

Multi-Tenant SaaS Platforms: Agar aap ek ai-chatbot bana rahe hain jo 100 alag-alag companies use karti hain, toh aap nahi chahenge ki Company A ka employee search kare toh Company B ka data leak ho jaye. Aap har document mein tenant_id store karte hain. Jab bhi koi employee query karega, system backend se automated filter laga dega: filter={"tenant_id": current_user.tenant_id}. Isse data security absolute 100% guarantee ho jati hai.

9. Interview questions

1. **Q: Pre-filtering aur Post-filtering mein kya difference hai? Kaunsa better hai?**
   A: Pre-filtering mein vector DB pehle metadata constraints run karke search space ko chota karta hai, fir similarity chalta hai (Highly accurate & efficient). Post-filtering mein pehle similarity se top-K nikalte hain fir metadata check karte hain (Isme danger hai ki accurate results drop ho jayein aur total results K se bohot kam bachein). Pre-filtering is always better.

2. **Q: Agar hume dynamically user ke natural language se filters extract karne hon (e.g., user says 'Show me 2025 HR docs'), toh hum LangChain mein kya use karenge?**
   A: Iske liye hum **Self-Query Retriever** use karte hain. Yeh ek aisa retriever hai jo pehle main LLM ko query bhejta hai, LLM query se structural filters aur semantic query alag karta hai, aur automatic filter format bana kar database ko bhejta hai.
3. **Q: Kya metadata filtering se vector search ki speed fast hoti hai?**
   A: Yes, agar proper indexing ho. Agar pre-filtering se 10,000 docs me se sirf 50 bachte hain, toh vector distance math sirf un 50 docs par chalega, jisse latency drastically decrease ho jati hai.

10. Advantages

- **Strict Accuracy:** Operational rules ko enforce karta hai (e.g., date restrictions, regional logic).
- **Security & Multi-tenancy:** Prevent karta hai unauthorized data leakage across departments or users.
- **Performance Boost:** Subsets par search karne se computation time save hota hai.

11. Limitations

- **Index Maintenance:** Agar aap hundreds of metadata fields create karenge, toh database indexing heavy ho jayegi aur storage cost badhegi.
- **Syntax Lock-in:** Chroma ka filter syntax alag hai, Pinecone ka alag, FAISS ka alag. Database switch karte waqt filters ka backend code rewrite karna padta hai.

12. Best practices

- **Normalize values:** Metadata values ko structured rakhein (e.g., dates ko hamesha string format YYYY-MM-DD ya epoch integers mein rakhein).
- **Don't over-index text:** Text paragraphs ko metadata fields mein mat dalo, metadata hamesha short strings, booleans, ya numeric tags hone chahiye.
- **Combine with Self-Query:** Agar user search query complex hai, toh explicit filters lagane ke bajaye LangChain ka SelfQueryRetriever integrate karein taaki user apne natural tareeqe se date/department filter apply kar sake.

Assignments & Practical Exercises

- **Exercise 1:** Ek loop banao jo dummy inputs mein 10 documents generate kare alag-alag file types ke (.pdf, .txt, .csv). Ek aisi query code karo jo sirf un documents ko search kare jo .pdf ya .txt hain (Hint: Use $or or $in operators depending on Vector DB syntax).
- **Assignment:** LangChain documentation se SelfQueryRetriever ka pattern padho aur explore karo ki kaise LLM automatic structural code likhta hai filters ke liye.

Common Mistakes Beginners Make

- *Type Mismatch:* Metadata mein year ko integer store kiya (2025) lekin search query filter

karte waqt string pass kar diya ("2025"). System zero results return karega aur aap pareshan ho jaoge ki code kyun nahi chal raha! Always match datatypes strictly.

Hamare saare fundamental blocks (Modules 1 to 8) ab crystal clear hain. Ab time aa gaya hai in sabhi components ko combine karke ek World-Class, Production-Grade Enterprise Architecture design karne ka.